



المحاضرة الثانية:

مقدمة في لغة C++

Introduction to C++

Topics for Lecture 2
2.1 The Parts of a C++ Program
2.2 The cout Object
2.3 The #include Directive
2.4 Variables, Literals, and Assignment Statements
2.5 Identifiers
2.6 Integer Data Types
2.7 The char Data Type
2.8 The C++ string Class
2.9 Floating-Point Data Types
2.10 The bool Data Type
2.11 Determining the Size of a Data Type
2.12 More about Variable Assignments and Initialization
2.13 Scope
2.14 Arithmetic Operators
2.15 Comments
2.16 Named Constants
2.17 Programming Style

2.1 The Parts of a C++ Program

المفهوم: برامج C++ تحتوي على أجزاء ومكونات تؤدي أغراضًا محددة.

كل برنامج C++ له هيكل تشريحي. على عكس التشريح البشري، أجزاء برنامج C++ ليست دائمًا في نفس المكان. ومع ذلك، فإن هذه الأجزاء موجودة، وخطوتك الأولى في تعلم C++ هي فهم ما هي هذه الأجزاء. سنبدأ بالنظر إلى البرنامج 1-2. باختصار:

- برامج C++ تتكون من أجزاء محددة، مثل الدوال والمتغيرات والتعليمات.
 - هذه الأجزاء تعمل معًا لتنفيذ المهام المطلوبة.
 - فهم هذه الأجزاء هو الأساس لتعلم كيفية كتابة برامج C++ بشكل فعال.
- سنستعرض البرنامج 1-2 لفهم هذه الأجزاء بشكل أفضل.

Program 2-1

```
1 // A simple C++ program
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << "Programming is great fun!";
8     return 0;
9 }
```

The output of the program is shown below. This is what appears on the screen when the program runs.

Program Output

Programming is great fun!

✓ لنفحص البرنامج سطرًا بسطر. إليك السطر الأول:

// A simple C++ program

الرمز // يحدد بداية تعليق (Comment). يتجاهل المترجم (Compiler) كل شيء من // حتى نهاية السطر. باختصار:

- التعليقات تُستخدم لإضافة ملاحظات أو توضيحات في الكود.
- لا تؤثر التعليقات على تنفيذ البرنامج لأن المترجم يتجاهلها.
- هذا السطر هو مجرد وصف للبرنامج ولا يؤثر على وظيفته.

✓ لنلق نظرة على السطر الثاني:

#include <iostream>

نظرًا لأن هذا السطر يبدأ بالرمز #، يُطلق عليه اسم توجيه المعالج المسبق (Preprocessor Directive). المعالج المسبق يقرأ البرنامج قبل أن يتم ترجمته (Compile) وينفذ فقط الأسطر التي تبدأ بالرمز #. يمكنك التفكير في المعالج المسبق كبرنامج "يجهز" الكود المصدري (Source Code) للمترجم.

توجيه `#include` يجعل المعالج المسبق يقوم بتضمين محتويات ملف آخر، يُعرف باسم ملف الرأس (Header File)، في البرنامج. يُسمى ملف الرأس بهذا الاسم لأنه يجب تضمينه في رأس (أو أعلى) البرنامج. الكلمة الموجودة بين القوسين `<>`، وهي `iostream`، هي اسم ملف الرأس الذي سيتم تضمينه. (اسم الملف هو `iostream`، والقوسان `<>` يشيران إلى أنه ملف رأس قياسي في C++).

ملف الرأس `<iostream>` يحتوي على كود يسمح لبرنامج C++ بعرض المخرجات على الشاشة وقراءة المدخلات من لوحة المفاتيح. نظرًا لأن هذا البرنامج يستخدم `cout` لعرض المخرجات على الشاشة، يجب تضمين ملف الرأس `<iostream>`. يتم تضمين محتويات ملف `<iostream>` في البرنامج عند النقطة التي يظهر فيها توجيه `#include`.

باختصار:

- `#include` يستخدم لتضمين ملفات الرأس في البرنامج.
- `<iostream>` هو ملف رأس قياسي في C++ يسمح بإدخال وإخراج البيانات.
- هذا السطر ضروري لاستخدام `cout` لعرض النصوص على الشاشة.

✓ لنلق نظرة على السطر الثالث:

`using namespace std;`

تحتوي البرامج عادةً على عدة عناصر ذات أسماء فريدة. في هذه المحاضرة، سنتعلم كيفية إنشاء المتغيرات. في محاضرات لاحقة سنتعلم إنشاء الدوال والكائنات. المتغيرات والدوال والكائنات هي أمثلة على كيانات البرنامج التي يجب أن يكون لها أسماء.

تستخدم C++ Namespaces لتنظيم أسماء كيانات البرنامج. العبارة `using namespace std;` تُعلن أن البرنامج سيتعامل مع كيانات تنتمي إلى Namespace المسماة `std` (نعم، لها أسماء!). السبب في حاجة البرنامج إلى الوصول إلى `namespace std` المسمى `std` هو أن كل الأسماء التي يتم إنشاؤها بواسطة ملف `<iostream>` هي جزء من هذا `namespace`. لكي يتمكن البرنامج من استخدام الكيانات الموجودة في `<iostream>`، يجب أن يكون لديه حق الوصول إلى `namespace std` المسمى `std`.

باختصار:

- `using namespace std;` تسمح للبرنامج باستخدام الكيانات القياسية في C++ دون الحاجة إلى تحديد `std` في كل مرة.
- هذا السطر يسهل الوصول إلى وظائف مثل `cout` و `cin` التي تنتمي إلى `std`.

✓ لنلق نظرة على السطر الخامس:

`int main ()`

هذا السطر يُعلن بداية دالة (Function). يمكن التفكير في الدالة كمجموعة من عبارات برمجية واحدة أو أكثر لها اسم مشترك. اسم هذه الدالة هو `main`، والأقواس `()` التي تتبع الاسم تشير إلى أنها دالة. الكلمة `int` تعني "عدد صحيح (Integer)"، وتشير إلى أن الدالة تُرجع قيمة عددية صحيحة إلى نظام التشغيل عند انتهاء تنفيذها.

على الرغم من أن معظم برامج C++ تحتوي على أكثر من دالة واحدة، إلا أن كل برنامج C++ يجب أن يحتوي على دالة تُسمى `main`. هذه الدالة هي نقطة البداية للبرنامج. إذا كنت تقرأ برنامج C++ لشخص آخر وتريد معرفة من أين يبدأ البرنامج، فقط ابحث عن الدالة المسماة `main`.

باختصار:

- `int main ()` هي الدالة الرئيسية التي يبدأ منها تنفيذ البرنامج.
- كل برنامج C++ يجب أن يحتوي على دالة `main`.

- الكلمة `int` تشير إلى أن الدالة تُرجع قيمة عددية صحيحة عند انتهائها.



NOTE: C++ is a case-sensitive language. That means it regards uppercase letters as being entirely different characters than their lowercase counterparts. In C++, the name of the function `main` must be written in all lowercase letters. C++ doesn't see "Main" the same as "main," or "INT" the same as "int." This is true for all the C++ key words.

✓ لنلق نظرة على السطر السادس:

```
{
```

هذا الرمز يُسمى القوس الافتتاحي `Left Brace` أو `Opening Brace`، وهو يرتبط ببداية الدالة `main`. جميع العبارات التي تشكل الدالة محاطة بمجموعة من الأقواس. إذا نظرت إلى السطر الثالث بعد القوس الافتتاحي، ستجد القوس الإغلاقي `}`. كل ما بين القوسين الافتتاحي والإغلاقي يُعتبر محتوى الدالة `main`. باختصار:

- `{` يحدد بداية كتلة الدالة `main`.
- جميع العبارات البرمجية الخاصة بالدالة يجب أن تكون محصورة بين `{` و `}`.
- هذا التنظيم يساعد في تحديد نطاق (Scope) الدالة وعباراتها.



WARNING! Make sure you have a closing brace for every opening brace in your program!

✓ لنلق نظرة على السطر السابع:

```
cout << "Programming is great fun!";
```

ببساطة، هذا السطر يعرض رسالة على الشاشة. سنتعلم المزيد عن `cout` وعامل الإدخال `>> cin` لاحقاً في هذه المحاضرة. الرسالة "Programming is great fun!" تُطبع بدون علامات الاقتباس. في مصطلحات البرمجة، مجموعة الأحرف الموجودة داخل علامات الاقتباس تُسمى نصاً ثابتاً (String Constant أو String Literal). باختصار:

- `cout` يُستخدم لعرض النصوص أو البيانات على الشاشة.
- `<<` هو عامل إدخال يُستخدم لإرسال البيانات إلى `cout`.
- النص الموجود داخل علامات الاقتباس " " يُسمى نصاً ثابتاً ويتم عرضه كما هو.



NOTE: This is the only line in the program that causes anything to be printed on the screen. The other lines, like `#include <iostream>` and `int main()`, are necessary for the framework of your program, but they do not cause any screen output. Remember, a program is a set of instructions for the computer. If something is to be displayed on the screen, you must use a programming statement for that purpose.

في نهاية السطر، نجد فاصلة منقوطة (Semicolon) تماماً كما تشير النقطة إلى نهاية الجملة في اللغة الإنجليزية، تشير الفاصلة المنقوطة إلى نهاية العبارة الكاملة في لغة C++.

- التعليقات (Comments): يتم تجاهلها من قبل المترجم، لذا لا تحتاج إلى فاصلة منقوطة في نهايتها.
- توجيهات المعالج المسبق (Preprocessor Directives): مثل عبارات `#include`، تنتهي بنهاية السطر ولا تحتاج أبداً إلى فاصلة منقوطة.

- بداية الدالة (Function Declaration): مثل `int main ()`، ليست عبارة كاملة، لذا لا توضع فاصلة منقوطة هنا أيضًا.

قد تبدو قواعد استخدام الفاصلة المنقوطة غير واضحة في البداية. بدلاً من القلق بشأنها الآن، ركز على تعلم أجزاء البرنامج. مع الممارسة، ستكتسب شعورًا طبيعيًا بأماكن وضع الفاصلة المنقوطة وأماكن عدم وضعها. باختصار:

- الفاصلة المنقوطة ; تُستخدم لإنهاء العبارات الكاملة في C++.
- لا تُستخدم مع التعليقات أو توجيهات المعالج المسبق أو بداية الدوال.
- مع الوقت، ستتعلم تلقائيًا أين يجب وضعها.

✓ لنلق نظرة على السطر الثامن:

`return 0;`

هذه العبارة تُرجع القيمة الصحيحة 0 إلى نظام التشغيل عند انتهاء تنفيذ البرنامج. القيمة 0 تشير عادةً إلى أن البرنامج تم تنفيذه بنجاح. باختصار:

- `return 0;` تُستخدم للإشارة إلى أن البرنامج انتهى بشكل صحيح.
- القيمة 0 هي قيمة تقليدية تُرجعها الدالة `main` للإشارة إلى نجاح التنفيذ.

✓ لنلق نظرة على السطر التاسع:

}

هذا القوس الإغلاقي } يحدد نهاية الدالة `main`. نظرًا لأن `main` هي الدالة الوحيدة في هذا البرنامج، فإن هذا القوس أيضًا يحدد نهاية البرنامج.

في البرنامج النموذجي، واجهت عدة مجموعات من الأحرف الخاصة. يقدم الجدول 1-2 ملخصًا موجزًا لكيفية استخدامها:

Table 2-1 Special Characters

Character	Name	Description
//	Double slash	Marks the beginning of a comment.
#	Pound sign	Marks the beginning of a preprocessor directive.
< >	Opening and closing brackets	Enclose a filename when used with the <code>#include</code> directive.
()	Opening and closing parentheses	Used in naming a function, as in <code>int main()</code> .
{ }	Opening and closing braces	Enclose a group of statements, such as the contents of a function.
" "	Opening and closing quotation marks	Enclose a string of characters, such as a message that is to be printed on the screen.
;	Semicolon	Marks the end of a complete programming statement.

باختصار:

- } يحدد نهاية الدالة `main` والبرنامج ككل.
- كل رمز أو حرف خاص في C++ له استخدام محدد يساعد في تنظيم الكود.

2.2 The cout Object

المفهوم: استخدم الكائن `cout` لعرض المعلومات على شاشة الكمبيوتر. في هذا القسم، ستتعلم كيفية كتابة برامج تعرض مخرجات على الشاشة. أبسط نوع من المخرجات التي يمكن للبرنامج عرضها هو المخرجات النصية (Console Output)، وهي مجرد نص عادي. كلمة `Console` هي مصطلح قديم في عالم الكمبيوتر. يعود أصلها إلى الأيام التي كان فيها مشغل الكمبيوتر يتفاعل مع النظام عن طريق الكتابة على جهاز طرفي (Terminal). كان هذا الجهاز، الذي يتكون من شاشة بسيطة ولوحة مفاتيح، يُعرف باسم `Console`. باختصار:

- `cout` هو كائن في C++ يُستخدم لعرض النصوص والبيانات على الشاشة.
- المخرجات النصية (Console Output) هي طريقة بسيطة لعرض المعلومات.
- مصطلح `Console` يشير إلى واجهة نصية للتفاعل مع الكمبيوتر.

Figure 2-1 A console window

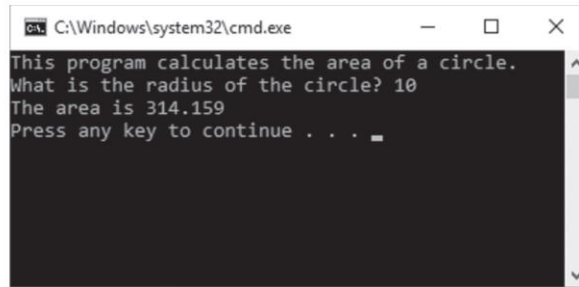


Figure 2-2 Think of << as an arrow pointing to cout

```
cout << "Programming is great fun!";
```

Think of the stream insertion operator as an arrow that points toward cout.

```
cout ← "Programming is great fun!";
```

Program 2-2

```
1 // A simple C++ program
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << "Programming is " << "great fun!";
8     return 0;
9 }
```

Program Output

```
Programming is great fun!
```

Program 2-3

```
1 // A simple C++ program
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << "Programming is ";
8     cout << "great fun!";
9     return 0;
10 }
```

Program Output

Programming is great fun!

Program 2-4

```
1 // An unruly printing program
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << "The following items were top sellers";
8     cout << "during the month of June:";
9     cout << "Computer games";
10    cout << "Coffee";
11    cout << "Aspirin";
12    return 0;
13 }
```

Program Output

The following items were top sellersduring the month of June:Computer
gamesCoffeeAspirin

Displaying a New Line:

There are two ways to instruct **cout** to start a new line. The first is to send **cout** a *stream manipulator* called **endl** (which is pronounced “end-line” or “end-L”). Program 2-5 is an example.

Program 2-5

```
1 // A well-adjusted printing program
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << "The following items were top sellers" << endl;
8     cout << "during the month of June:" << endl;
```

Program 2-5 (continued)

```
9     cout << "Computer games" << endl;
10    cout << "Coffee" << endl;
11    cout << "Aspirin" << endl;
12    return 0;
13 }
```

Program Output

```
The following items were top sellers
during the month of June:
Computer games
Coffee
Aspirin
```



NOTE: The last character in `endl` is the lowercase letter L, *not* the number one.

Another way to cause **cout** to go to a new line is to insert an *escape sequence* in the string itself. An escape sequence starts with the backslash character (\) and is followed by one or more control characters. It allows you to control the way output is displayed by embedding commands within the string itself. Program 2-6 is an example.

Program 2-6

```
1 // Yet another well-adjusted printing program
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << "The following items were top sellers\n";
8     cout << "during the month of June:\n";
9     cout << "Computer games\nCoffee";
10    cout << "\nAspirin\n";
11    return 0;
12 }
```

Program Output

```
The following items were top sellers
during the month of June:
Computer games
Coffee
Aspirin
```


Common Mistakes:

<pre>// Error! cout << "Four Score/nAnd seven/nYears ago./n";</pre>
<pre>// Error! This code will not compile. cout << "Good" << \n; cout << "Morning" << \n;</pre>
<pre>// This will work. cout << "Good\n"; cout << "Morning\n";</pre>

Table 2-2 Common Escape Sequences

Escape Sequence	Name	Description
\n	Newline	Causes the cursor to go to the next line for subsequent printing.
\t	Horizontal tab	Causes the cursor to skip over to the next tab stop.
\a	Alarm	Causes the computer to beep.
\b	Backspace	Causes the cursor to back up, or move left one position.
\r	Return	Causes the cursor to go to the beginning of the current line, not the next line.
\\	Backslash	Causes a backslash to be printed.
\'	Single quote	Causes a single quotation mark to be printed.
\"	Double quote	Causes a double quotation mark to be printed.



WARNING! When using escape sequences, do not put a space between the backslash and the control character.

For example, consider the following string literal:

"One\nTwo\nThree\n"

The diagram in Figure 2-3 breaks this string into its individual characters. Notice how each of the \n escape sequences are considered one character.

Figure 2-3 Individual characters in a string

O n e \n T w o \n T h r e e \n

Raw String Literals

In C++ a raw string literal begins with the characters R" (and ends with the characters)". Here is an example: **R"(One\nTwo\nThree)"**

Program 2-7

```
1 // This program demonstrates a raw string.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << "Here is the raw string:" << endl;
8     cout << R"(One\nTwo\nThree)" << endl << endl;
9     cout << "Here is the regular string:" << endl;
10    cout << "One\nTwo\nThree" << endl;
11    return 0;
12 }
```

Program Output

```
Here is the raw string:
One\nTwo\nThree
Here is the regular string:
One
Two
Three
```

2.3 The # include Directive

المفهوم: توجيه #include يتسبب في إدراج محتويات ملف آخر داخل البرنامج.

السطر التالي ظهر بالقرب من أعلى كل برنامج مثال:

```
#include <iostream>
```

يجب تضمين ملف الرأس iostream في أي برنامج يستخدم الكائن cout وذلك لأن cout ليس جزءًا من "النواة الأساسية" للغة C++ ، بل هو جزء من مكتبة تدفق الإدخال والإخراج. يحتوي ملف الرأس iostream على معلومات تصف كائنات iostream. بدونها، لن يعرف المترجم كيفية ترجمة البرنامج الذي يستخدم cout بشكل صحيح.

توجيهات المعالج المسبق (Preprocessor directives) ليست عبارات في لغة C++. إنها أوامر تُرسل إلى المعالج المسبق، الذي يعمل قبل المترجم (ومن هنا جاء اسم "المعالج المسبق"). مهمة المعالج المسبق هي إعداد البرامج بطريقة تجعل حياة المبرمج أسهل.



WARNING! Do not put semicolons at the end of processor directives. Because preprocessor directives are not C++ statements, they do not require semicolons. In many cases, an error will result from a preprocessor directive terminated with a semicolon.

2.4 Variables, Literals, and Assignment Statements

المفهوم: تمثل المتغيرات مواقع تخزين في ذاكرة الكمبيوتر. بينما الثوابت (Literals) هي قيم ثابتة يتم تعيينها للمتغيرات.

تتيح لك المتغيرات تخزين البيانات والتعامل معها في ذاكرة الكمبيوتر. توفر المتغيرات "واجهة" للوصول إلى ذاكرة الوصول العشوائي (RAM). جزء من مهمة البرمجة هو تحديد عدد المتغيرات التي سيحتاجها البرنامج وأنواع البيانات التي ستخزنها. البرنامج 2-8 هو مثال على برنامج C++ يحتوي على متغير. لاحظ السطر 7:

```
int number;
```

هذا يسمى **تعريف المتغير**. وهو يخبر المترجم باسم المتغير ونوع البيانات التي سيحتويها. هذا السطر يشير إلى أن اسم المتغير هو `number`. الكلمة `int` تعني عددًا صحيحًا (integer)، لذلك سيتم استخدام `number` فقط لتخزين الأعداد الصحيحة. لاحقًا في هذه المحاضرة، سنتعلم جميع أنواع البيانات التي تدعمها لغة C++.

Program 2-8

```
1 // This program has a variable.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int number;
8
9     number = 5;
10    cout << "The value in number is " << number << endl;
11    return 0;
12 }
```

Program Output

The value in number is 5

لاحظ أن تعريفات المتغيرات تنتهي بفاصلة منقوطة (;). الآن انظر إلى السطر 9:

```
number = 5;
```

هذا يسمى تعيين قيمة أو إسناد قيمة (Assignment). علامة المساواة (=) هي عامل يقوم بنسخ القيمة الموجودة على يمينه (وهي 5 في هذه الحالة) إلى المتغير الموجود على يساره (وهو number). بعد تنفيذ هذا السطر، سيتم تعيين القيمة 5 للمتغير number.



NOTE: This line does not print anything on the computer's screen. It runs silently behind the scenes, storing a value in RAM.

انظر إلى السطر 10:

```
cout << "The value in number is " << number << endl;
```

العنصر الثاني المرسل إلى cout هو اسم المتغير number. عندما ترسل اسم متغير إلى cout، فإنه يطبع محتويات هذا المتغير. لاحظ أنه لا توجد علامات اقتباس حول number. انظر إلى ما يحدث في البرنامج 2-9.

شرح:

1. العنصر الأول "The value in number is " :
هذا نص ثابت (string literal) سيتم طباعته كما هو.
2. العنصر الثاني number :
هذا هو المتغير الذي يحتوي على القيمة 5 (تم تعيينها في السطر السابق). عندما يتم إرسال المتغير إلى cout، يتم طباعة القيمة المخزنة فيه، وهي 5.
3. العنصر الثالث endl :
هذا يؤدي إلى إنهاء السطر الحالي والانتقال إلى سطر جديد.

الناتج المتوقع:

The value in number is 5

ملاحظة:

إذا وضعت علامات اقتباس حول number مثل "number"، فسيتم طباعة الكلمة "number" بدلاً من قيمة المتغير. على سبيل المثال:

```
cout << "The value in number is " << "number" << endl;
```

الناتج في هذه الحالة:

The value in number is number

لذلك، تأكد من عدم وضع علامات اقتباس حول اسم المتغير إذا كنت تريد طباعة قيمته وليس اسمه.

Program 2-9

```
1 // This program has a variable.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int number;
8
9     number = 5;
10    cout << "The value in number is " << "number" << endl;
11    return 0;
12 }
```

Program Output

The value in number is number

البرنامج 10-2 يوضح مثالاً يستخدم متغيراً و عدة ثوابت (literals) لنلقي نظرة على كيفية عمل هذا البرنامج:

مثال البرنامج 10-2:

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int apples;
    apples = 5;
    cout << "I have " << apples << " apples." << endl;
    cout << "You have " << 3 << " apples." << endl;
    cout << "We have a total of " << apples + 3 << " apples." << endl;
    return 0;
}
```

شرح البرنامج:

1. تعريف المتغير:

```
int apples;
```

هنا قمنا بتعريف متغير اسمه apples من نوع int (عدد صحيح).

2. تعيين قيمة للمتغير:

```
apples = 5;
```

هنا قمنا بتعيين القيمة 5 للمتغير apples.

3. طباعة النتائج:

○ السطر الأول:

```
cout << "I have " << apples << " apples." << endl;
```

هنا نطبع النص "I have " متبوعاً بقيمة المتغير apples وهي 5، ثم النص ". apples." فيكون الناتج:

I have 5 apples.

○ السطر الثاني:

```
cout << "You have " << 3 << " apples." << endl;
```

هنا نطبع النص "You have " متبوعاً بالثابت 3، ثم النص ". apples." فيكون الناتج:

You have 3 apples.

○ السطر الثالث:

```
cout << "We have a total of " << apples + 3 << " apples." << endl;
```

هنا نطبع النص "We have a total of " متبوعاً بحاصل جمع قيمة المتغير apples وهي 5 والثابت 3، ثم النص ". apples." فيكون الناتج:

We have a total of 8 apples.

الناتج الكامل للبرنامج:

I have 5 apples.

You have 3 apples.

We have a total of 8 apples.

ملاحظات:

- المتغيرات: تُستخدم لتخزين البيانات التي يمكن تغييرها أثناء تنفيذ البرنامج مثل apples
 - الثوابت (Literals): هي قيم ثابتة تُكتب مباشرة في الكود مثل 5 و 3
 - الجمع بين المتغيرات والثوابت: يمكن إجراء عمليات حسابية بين المتغيرات والثوابت مثل apples + 3
- هذا البرنامج يوضح كيفية استخدام المتغيرات والثوابت معاً لإنشاء ناتج ديناميكي ومفيد.



NOTE: Literals are also called constants.

2.5 Identifiers

المفهوم: اختر أسماء متغيرات تشير إلى الغرض من استخدامها.

المُعَرِّف (Identifier) هو اسم يحدده المبرمج لتمثيل عنصر معين في البرنامج. أسماء المتغيرات هي أمثلة على المُعرفات. يمكنك اختيار أسماء المتغيرات الخاصة بك في لغة ++C ، طالما أنك لا تستخدم أيًا من الكلمات المفتاحية (Key Words) الخاصة بـ ++C . هذه الكلمات المفتاحية تشكل "النواة الأساسية" للغة ولديها أغراض محددة. يوضح الجدول 4-2 قائمة كاملة بالكلمات المفتاحية في ++C لاحظ أنها جميعًا مكتوبة بأحرف صغيرة (lowercase).

Table 2-4 The C++ Key Words

alignas	const	for	private	throw
alignof	constexpr	friend	protected	true
and	const_cast	goto	public	try
and_eq	continue	if	register	typedef
asm	decltype	inline	reinterpret_cast	typeid
auto	default	int	return	typename
bitand	delete	long	short	union
bitor	do	mutable	signed	unsigned
bool	double	namespace	sizeof	using
break	dynamic_cast	new	static	virtual
case	else	noexcept	static_assert	void
catch	enum	not	static_cast	volatile
char	explicit	not_eq	struct	wchar_t
char16_t	export	nullptr	switch	while
char32_t	extern	operator	template	xor
class	false	or	this	xor_eq
compl	float	or_eq	thread_local	

المُعَرِّفات الصحيحة (Legal Identifiers):

بغض النظر عن النمط الذي تتبناه، كن متسقًا واجعل أسماء المتغيرات منطقية قدر الإمكان. فيما يلي بعض القواعد المحددة التي يجب اتباعها مع جميع المُعرفات:

1. **الحرف الأول:** يجب أن يكون أحد الأحرف من a إلى z، أو من A إلى Z، أو رمز الشرطة السفلية (_). (Under Score).
2. **بعد الحرف الأول:** يمكنك استخدام الأحرف من a إلى z أو من A إلى Z، أو الأرقام من 0 إلى 9، أو رمز الشرطة السفلية (_).
3. **التمييز بين الأحرف الكبيرة والصغيرة:** الأحرف الكبيرة والصغيرة تعتبر مختلفة. هذا يعني أن ItemsOrdered ليست نفسها itemsordered.

أمثلة على أسماء المتغيرات صحيحة أو غير صحيحة في لغة C++:

السبب	الحالة	اسم المتغير
يتبع القواعد المذكورة أعلاه.	صحيح	itemsOrdered
يتبع القواعد المذكورة أعلاه.	صحيح	items_ordered
يتبع القواعد المذكورة أعلاه.	صحيح	ItemsOrdered
يحتوي على (-) غير مسموح بها.	غير صحيح	items-ordered
يبدأ برقم.	غير صحيح	1stOrder
يبدأ بحرف ويحتوي على رقم.	صحيح	order1
يبدأ بشرطة سفلية (_)	صحيح	_order
يحتوي على رمز غير مسموح به (!)	غير صحيح	order!
كلمة مفتاحية محجوزة في C++	غير صحيح	int

ملاحظات:

- يجب أن تكون أسماء المتغيرات واضحة وذات معنى لتسهيل فهم الكود.
- تجنب استخدام الكلمات المفتاحية أو أسماء غير واضحة.
- الالتزام بالقواعد المذكورة يضمن كتابة كود صحيح وسهل الصيانة.

2.6 Integer Data Types

المفهوم: هناك العديد من أنواع البيانات المختلفة. يتم تصنيف المتغيرات وفقاً لنوع البيانات الخاص بها، وهو ما يحدد نوع المعلومات التي يمكن تخزينها فيها. المتغيرات من نوع الأعداد الصحيحة (Integer) يمكنها فقط تخزين الأعداد الكاملة (بدون كسور).

اعتباراتك الرئيسية عند اختيار نوع البيانات الرقمية هي:

- أكبر وأصغر الأرقام التي يمكن تخزينها في المتغير.
 - مقدار الذاكرة التي يستخدمها المتغير.
 - ما إذا كان المتغير يخزن أرقاماً ذات إشارة (signed) أو بدون إشارة (unsigned).
 - عدد الأرقام العشرية (الدقة) التي يدعمها المتغير.
- حجم المتغير** هو عدد وحدات البايت (bytes) التي يستخدمها في الذاكرة. بشكل عام، كلما كان المتغير أكبر، كلما زاد النطاق الذي يمكنه تخزينه.
- يوضح الجدول 2-6 أنواع البيانات الصحيحة (Integer) في لغة C++ مع أحجامها النموذجية والنطاقات التي تدعمها.



NOTE: The data type sizes and ranges shown in Table 2-6 are typical on many systems. Depending on your operating system, the sizes and ranges may be different.

Table 2-6 Integer Data Types

Data Type	Typical Size	Typical Range
short int	2 bytes	−32,768 to +32,767
unsigned short int	2 bytes	0 to +65,535
int	4 bytes	−2,147,483,648 to +2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long int	4 bytes	−2,147,483,648 to +2,147,483,647
unsigned long int	4 bytes	0 to 4,294,967,295
long long int	8 bytes	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long int	8 bytes	0 to 18,446,744,073,709,551,615

ملاحظات:

- الأرقام ذات الإشارة: **(Signed)** يمكنها تخزين قيم موجبة وسالبة.
 - الأرقام بدون إشارة: **(Unsigned)** يمكنها تخزين قيم موجبة فقط.
 - اختيار النوع المناسب يعتمد على احتياجات البرنامج من حيث النطاق والدقة واستخدام الذاكرة.
- أمثلة على تعريف المتغيرات:

```
int days;
unsigned int speed;
short int month;
unsigned short int amount;
long int deficit;
unsigned long int insects;
```

الاختصارات لأنواع البيانات:

يمكن اختصار أنواع البيانات في الجدول 2-6 باستثناء int كما يلي:

- short int → short.
- unsigned short int → unsigned short.
- unsigned int → unsigned.
- long int → long.
- unsigned long int → unsigned long.
- long long int → long long.
- unsigned long long int → unsigned long long.

استخدام الاختصارات:

نظرًا لأنها تُبسّط عبارات التعريف، يستخدم المبرمجون عادةً أسماء أنواع البيانات المختصرة. إليك بعض الأمثلة:

unsigned speed;

short month;

unsigned short amount;

long deficit;

unsigned long insects;

long long grandTotal;

unsigned long long lightYearDistance;

ملاحظات:

- الاختصارات تجعل الكود أكثر وضوحًا وسهولة في القراءة.
- يجب اختيار نوع البيانات المناسب بناءً على نطاق القيم المطلوبة وكفاءة استخدام الذاكرة.



NOTE: The long long int and the unsigned long long int data types were introduced in C++ 11.

انظر البرنامجين التاليين:

Program 2-11

```
1 // This program has variables of several of the integer types.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int checking;
8     unsigned int miles;
9     long diameter;
10
11     checking = -20;
12     miles = 4276;
13     diameter = 100000;
14     cout << "We have made a long journey of " << miles;
15     cout << " miles.\n";
16     cout << "Our checking account balance is " << checking;
17     cout << "\nThe galaxy is about " << diameter;
18     cout << " light years in diameter.\n";
19     return 0;
20 }
```

Program Output

```
We have made a long journey of 4276 miles.
Our checking account balance is -20
The galaxy is about 100000 light years in diameter.
```

Program 2-12

```
1 // This program shows three variables defined on the same line.
2 #include <iostream>
3 using namespace std;
```

(program continues)

Program 2-12

(continued)

```
4
5 int main()
6 {
7     int floors, rooms, suites;
8
9     floors = 15;
10    rooms = 300;
11    suites = 30;
12    cout << "The Grande Hotel has " << floors << " floors\n";
13    cout << "with " << rooms << " rooms and " << suites;
14    cout << " suites.\n";
15    return 0;
16 }
```

Program Output

The Grande Hotel has 15 floors
with 300 rooms and 30 suites.

Using Digit Separators

على سبيل المثال، بافتراض أن المتغير `amount` تم تعريفه كـ `int`، انظر إلى العبارة التالية:

```
amount = 1'000'000;
```

مترجم لغة C++ يتجاهل علامات الفواصل المفردة (') التي تظهر في الرقم. هذه العبارة تقوم بتعيين القيمة `1000000` إلى المتغير `amount`.

شرح:

- الغرض من علامات الفواصل (') تُستخدم لجعل الأرقام الكبيرة أكثر قابلية للقراءة من قبل المبرمجين. على سبيل المثال، الرقم `1'000'000` أسهل في القراءة من `1000000`.
- تجاهل المترجم لعلامات الفواصل: عند تنفيذ الكود، يتجاهل المترجم هذه العلامات ويعامل الرقم كـ `1000000`.
- نتيجة العبارة: يتم إسناد القيمة `1000000` إلى المتغير `amount`.

مثال آخر:

```
int population = 7'800'000'000;
```

هنا، يتم تعيين (إسناد) القيمة `7800000000` إلى المتغير `population`، لكن استخدام علامات الفواصل يجعل الرقم أكثر وضوحًا.



TIP: When writing long integer literals or long long integer literals, you can use either an uppercase or a lowercase L. Because the lowercase l looks like the number 1, you should always use the uppercase L.

مثال:

```
long amount;
amount = 32L;
```

```
long long amount;
amount = 32LL;
```

If You Plan to Continue in Computer Science: Binary, Hexadecimal, and Octal Literals:

عادة ما يعبر المبرمجون عن القيم بأنظمة ترقيم غير النظام العشري (أو الأساس 10). أنظمة مثل الثنائي (الأساس 2)، والستة عشري (الأساس 16)، والثماني (الأساس 8) شائعة لأنها تجعل بعض مهام البرمجة أكثر ملاءمة مقارنة بالأرقام العشرية.

بشكل افتراضي، يفترض C++ أن جميع الثوابت الرقمية الصحيحة مكتوبة بالنظام العشري. يمكنك التعبير عن الأرقام الثنائية بوضع 0b أمامها (هذا صفر ثم حرف b، وليس حرف o ثم b) إليك كيف يُكتب الرقم الثنائي 1101 في C++:

```
0b1101
```

يمكنك التعبير عن الأرقام الستة عشرية بوضع 0x أمامها (صفر ثم حرف x، وليس حرف o ثم x) إليك كيف يُكتب الرقم الستة عشري F4 في C++:

```
0xF4
```

أما الأرقام الثمانية فيجب أن تسبقها (0 صفر، وليس حرف o) على سبيل المثال، الرقم الثماني 31 يُكتب كالتالي:

```
031
```

ملاحظات:

- النظام الثنائي (Binary): يستخدم في البرمجة المنخفضة المستوى (low-level programming) والعمليات على البتات (bitwise operations).
 - النظام الستة عشري (Hexadecimal): شائع في تمثيل الذاكرة والعناوين.
 - النظام الثماني (Octal): أقل شيوعًا ولكنه ما زال مستخدمًا في بعض السياقات.
- هذه الأنظمة توفر طرقًا مختلفة لتمثيل الأرقام، مما يجعلها مفيدة في تطبيقات محددة.

2.7 The char Data Type

يستخدم هذا النمط لتخزين محارف منفصلة، حيث يتم تخزين حرف واحد فقط. كيفية التصريح عن هذا النمط والإسناد:

```
char letter;
letter = 'g';
```

لاحظ أنه تم إحاطة المحرف بعلامة اقتباس مفردة (single quotation mark) ومن ثم الإسناد إلى متغير من نوع (char).

```
letter = "g"; // ERROR! Cannot assign a string to a char
```

النمط من نوع (char) حجمه (1 byte) في الذاكرة (هذا يتعلق بالنظام الذي يتم العمل عليه).
البرنامج 2-13 يبين كيفية التعامل مع المحارف:

Program 2-13

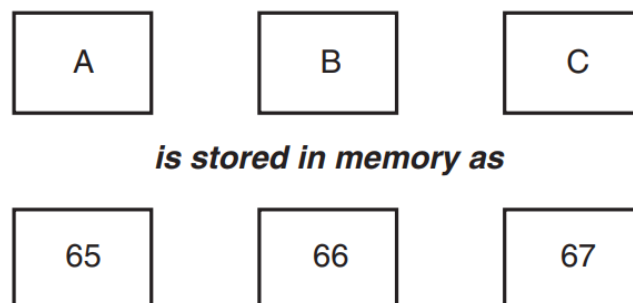
```
1 // This program works with characters.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     char letter;
8
9     letter = 'A';
10    cout << letter << endl;
11    letter = 'B';
12    cout << letter << endl;
13    return 0;
14 }
```

Program Output

A
B

إن الطريقة الأكثر شيوعاً لترميز المحارف هي (ASCII) والتي هي اختصار (American Standard Code for Information Interchange). حيث يتم إعطاء رقم فريد لكل محرف وعد تخزين المحرف في الذاكرة يُخزن بالرمز العددي الموافق له وعند إعطاء الحاسوب الأمر بطباعة العدد على الشاشة فإنه يطبعه بالشكل الحرفي المعروف.

Figure 2-4 Characters and their ASCII codes



نورد فيما يلي جدولاً بالأحرف الطباعية وفق المعيار (ASCII):

Nonprintable ASCII Characters				Printable ASCII Characters			
Dec	Hex	Oct	Name of Character	Dec	Hex	Oct	Character
0	0	0	NULL	32	20	40	(Space)
1	1	1	SOTT	33	21	41	!
2	2	2	STX	34	22	42	"
3	3	3	ETY	35	23	43	#
4	4	4	EOT	36	24	44	\$
5	5	5	ENQ	37	25	45	%
6	6	6	ACK	38	26	46	&
7	7	7	BELL	39	27	47	'
8	8	10	BKSPC	40	28	50	(
9	9	11	HZTAB	41	29	51	)
10	a	12	NEWLN	42	2a	52	*
11	b	13	VTAB	43	2b	53	+
12	c	14	FF	44	2c	54	,
13	d	15	CR	45	2d	55	-
14	e	16	SO	46	2e	56	.
15	f	17	SI	47	2f	57	/
16	10	20	DLE	48	30	60	0
17	11	21	DC1	49	31	61	1
18	12	22	DC2	50	32	62	2
19	13	23	DC3	51	33	63	3
20	14	24	DC4	52	34	64	4
21	15	25	NAK	53	35	65	5
22	16	26	SYN	54	36	66	6
23	17	27	ETB	55	37	67	7
24	18	30	CAN	56	38	70	8
25	19	31	EM	57	39	71	9
26	1a	32	SUB	58	3a	72	:
27	1b	33	ESC	59	3b	73	;
28	1c	34	FS	60	3c	74	<
29	1d	35	GS	61	3d	75	=
30	1e	36	RS	62	3e	76	>
31	1f	37	US	63	3f	77	?
127	7f	177	DEL	64	40	100	@

Printable ASCII Characters				Printable ASCII Characters			
Dec	Hex	Oct	Character	Dec	Hex	Oct	Character
65	41	101	A	96	60	140	`
66	42	102	B	97	61	141	a
67	43	103	C	98	62	142	b
68	44	104	D	99	63	143	c
69	45	105	E	100	64	144	d
70	46	106	F	101	65	145	e
71	47	107	G	102	66	146	f
72	48	110	H	103	67	147	g
73	49	111	I	104	68	150	h
74	4a	112	J	105	69	151	i
75	4b	113	K	106	6a	152	j
76	4c	114	L	107	6b	153	k
77	4d	115	M	108	6c	154	l
78	4e	116	N	109	6d	155	m
79	4f	117	O	110	6e	156	n
80	50	120	P	111	6f	157	o
81	51	121	Q	112	70	160	p
82	52	122	R	113	71	161	q
83	53	123	S	114	72	162	r
84	54	124	T	115	73	163	s
85	55	125	U	116	74	164	t
86	56	126	V	117	75	165	u
87	57	127	W	118	76	166	v
88	58	130	X	119	77	167	w
89	59	131	Y	120	78	170	x
90	5a	132	Z	121	79	171	y
91	5b	133	[	122	7a	172	z
92	5c	134	\	123	7b	173	{
93	5d	135	]	124	7c	174	
94	5e	136	^	125	7d	175	}
95	5f	137	_	126	7e	176	~

البرنامج 2-14 يبين أن التعامل مع المحارف في الحقيقة هو التعامل مع أرقام:

Program 2-14

```
1 // This program demonstrates the close relationship between
2 // characters and integers.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     char letter;
9
10    letter = 65;
11    cout << letter << endl;
12    letter = 66;
13    cout << letter << endl;
```

(program continues)

Program 2-14 (continued)

```
14    return 0;
15 }
```

Program Output

A
B

The Difference between String Literals and Character Literals

من المهم عدم الخلط بين المحرف (character) والسلسلة النصية (string)، حيث أن السلسلة النصية هي تتالي عدد من المحارف تُخزن في الذاكرة بمواقع متتالية ومن حيث المبدأ يمكن أن تأخذ أي طول. في لغة C++ يتم إلحاق بايت إضافي لنهاية السلسلة النصية لدى تخزينها في الذاكرة مبيناً نهاية السلسلة النصية يدعى (null terminator or null character) كما هو مبين في الشكل 2-5:

Figure 2-5 The string literal "Sebastian"

S	e	b	a	s	t	i	a	n	\0
---	---	---	---	---	---	---	---	---	----

نلاحظ أن طول السلسلة هو 9 ولكن يتم تخزينها في الذاكرة بحجم 10 بايت.



NOTE: C++ automatically places the null terminator at the end of string literals.

لاحظ كيفية تخزين الحرف والسلسلة النصية في الذاكرة:

Figure 2-6 'A' as a character and "A" as a string

'A' is stored as

A

"A" is stored as

A	\0
---	----

Figure 2-7 ASCII codes

'A' is stored as

65

"A" is stored as

65	0
----	---

كما ترى 'A' بحجم 1 بايت ولكن "A" بحجم 2 بايت لأن المحرف يتم تخزينه في الذاكرة طبقاً للكود ASCII. لذلك فإن التخزين الحقيقي في الذاكرة يتم وفق الشكل 2-7. على سبيل المثال في الرماز التالي لاحظ الفرق:

```
char letter;
```

```
letter = 'A'; // This will work.
```

```
letter = "A"; // This will not work!
```

لاحظ في البرنامج 2-15 أنه يتم التعامل مع \n كمحرف واحد لذلك يتم استخدام علامة الاقتباس المفردة.

Program 2-15

```
1 // This program uses character literals.
2 #include <iostream>
3 using namespace std;
```

Program 2-15 (continued)

```
4
5 int main()
6 {
7     char letter;
8
9     letter = 'A';
10    cout << letter << '\n';
11    letter = 'B';
12    cout << letter << '\n';
13    return 0;
14 }
```

Program Output

```
A
B
```


Let's review some important points regarding characters and strings:

- Printable characters are internally represented by numeric codes. Most computers use ASCII codes for this purpose.
- Characters normally occupy a single byte of memory.
- Strings are consecutive sequences of characters that occupy consecutive bytes of memory.
- String literals are always stored in memory with a null terminator at the end. This marks the end of the string.
- Character literals are enclosed in single quotation marks.
- String literals are enclosed in double quotation marks.
- Escape sequences such as '\n' are stored internally as a single character.

2.8 The C++ string Class

المفهوم: إن C++ المعيارية لديها نمط بيانات خاص للتعامل مع السلاسل النصية وتخزينها.

لاحظ البرنامج التالي 2-16 الذي يبين كيفية الاستخدام حيث يتم التعامل مع السلسلة النصية كالتعامل مع المتغير:

Program 2-16

```
1 // This program demonstrates the string class.
2 #include <iostream>
3 #include <string> // Required for the string class.
4 using namespace std;
5
6 int main()
7 {
8     string movieTitle;
9
10    movieTitle = "Wheels of Fury";
11    cout << "My favorite movie is " << movieTitle << endl;
12    return 0;
13 }
```

Program Output

My favorite movie is Wheels of Fury

2.9 Floating-Point Data Types

المفهوم: يتم استخدام أنماط بيانات الفاصلة العشرية لتعريف متغيرات يتم إسنادها بأعداد حقيقية.

يتم تخزين الأرقام العشرية داخلياً بشكل مشابه للنمط العلمي كما في الجدول التالي:

When a floating-point number is stored in memory, it is stored as the mantissa and the power of 10.

Ex: 47,281.97 → In scientific notation: 4.728197×10^4 → In E notation:

4.728197 *E* ⁴
mantessa *power of 10*

Table 2-7 Floating-Point Representations

Decimal Notation	Scientific Notation	E Notation
247.91	2.4791×10^2	2.4791E2
0.00072	7.2×10^{-4}	7.2E-4
2,900,000	2.9×10^6	2.9E6

Table 2-8 shows the typical sizes and ranges of floating-point data types.

Table 2-8 Floating-Point Data Types on PCs

Data Type	Key Word	Description
Single precision	float	4 bytes. Numbers between $\pm 3.4\text{E}-38$ and $\pm 3.4\text{E}38$
Double precision	double	8 bytes. Numbers between $\pm 1.7\text{E}-308$ and $\pm 1.7\text{E}308$
Long double precision	long double	8 bytes*. Numbers between $\pm 1.7\text{E}-308$ and $\pm 1.7\text{E}308$

*Some compilers use 10 bytes for long doubles. This allows a range of $\pm 3.4\text{E}-4932$ to $\pm 1.1\text{E}4832$.

انظر إلى البرنامج 2-17 ولاحظ كيفية التعامل مع الأعداد العشرية:

Program 2-17

```
1 // This program uses floating-point data types.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     float distance;
8     double mass;
9
10    distance = 1.495979E11;
11    mass = 1.989E30;
12    cout << "The Sun is " << distance << " meters away.\n";
13    cout << "The Sun's mass is " << mass << " kilograms.\n";
14    return 0;
15 }
```

Program Output

```
The Sun is 1.49598e+011 meters away.
The Sun's mass is 1.989e+030 kilograms.
```

All of the following floating-point literals are equivalent:

1.4959E11

1.4959e11

1.4959E + 11

1.4959e + 11

149590000000.00



NOTE: Because floating-point literals are normally stored in memory as doubles, most compilers issue a warning message when you assign a floating-point literal to a float variable. For example, assuming num is a float, the following statement might cause the compiler to generate a warning message:

```
num = 14.725;
```

You can suppress the warning message by appending the f suffix to the floating-point literal, as shown below:

```
num = 14.725f;
```

If you want to force a value to be stored as a long double, append an L or l to it, as in the following examples:

1034.56L

89.2l

The compiler won't confuse these with long integers because they have decimal points. (Remember, the lowercase L looks so much like the number 1 that you should always use the uppercase L when suffixing literals.)

Assigning Floating-Point Values to Integer Variables

When a floating-point value is assigned to an integer variable, the fractional part of the value (the part after the decimal point) is discarded. For example, look at the following code:

```
int number;
```

```
number = 7.5; // Assigns 7 to number (Truncation not Rounding)
```



NOTE: When a floating-point value is truncated, it is not rounded. Assigning the value 7.9 to an int variable will result in the value 7 being stored in the variable.



WARNING! Floating-point variables can hold a much larger range of values than integer variables can. If a floating-point value is being stored in an integer variable, and the whole part of the value (the part before the decimal point) is too large for the integer variable, an invalid value will be stored in the integer variable.

2.10 The bool Data Type

المفهوم: يتم تعيين المتغيرات المنطقية (Boolean) إما إلى "صحيح" (true) أو "خطأ" (false) "التعبيرات التي يكون لها قيمة "صحيح" أو "خطأ" تُسمى تعبيرات منطقية (Boolean expressions)، وقد سُميت بهذا الاسم تكريماً لعالم الرياضيات الإنجليزي جورج بول (1815–1864).

نوع البيانات bool يسمح لك بإنشاء متغيرات صغيرة من نوع عدد صحيح تكون مناسبة لحفظ قيم "صحيح" أو "خطأ". يوضح البرنامج 2-18 تعريف وتعيين متغير من نوع bool.

Program 2-18

```
1 // This program demonstrates boolean variables.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     bool boolValue;
8
9     boolValue = true;
10    cout << boolValue << endl;
11    boolValue = false;
12    cout << boolValue << endl;
```

Program 2-18 (continued)

```
13    return 0;
14 }
```

Program Output

```
1
0
```

كما ترى من ناتج البرنامج، يتم تمثيل القيمة "صحيح" (true) في الذاكرة بالرقم 1، بينما يتم تمثيل القيمة "خطأ" (false) بالرقم 0.

2.11 Determining the Size of a Data Type

المفهوم: يمكن استخدام العامل sizeof لتحديد حجم نوع بيانات معين على أي نظام.

عامل خاص يُسمى sizeof يقوم بالإبلاغ عن عدد البايتات المستخدمة في الذاكرة لأي نوع بيانات أو متغير. يوضح البرنامج 2-19 كيفية استخدامه.

Program 2-19

```
1 // This program determines the size of integers, long
2 // integers, and long doubles.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     long double apple;
9
10    cout << "The size of an integer is " << sizeof(int);
11    cout << " bytes.\n";
12    cout << "The size of a long integer is " << sizeof(long);
13    cout << " bytes.\n";
14    cout << "An apple can be eaten in " << sizeof(apple);
15    cout << " bytes!\n";
16    return 0;
17 }
```

Program Output

```
The size of an integer is 4 bytes.
The size of a long integer is 4 bytes.
An apple can be eaten in 8 bytes!
```

2.12 More about Variable Assignments and Initialization

المفهوم: عملية الإسناد (assignment) نقوم بتعيين أو نسخ قيمة إلى متغير. عندما يتم تعيين قيمة لمتغير كجزء من تعريف المتغير، تُسمى هذه العملية التهيئة (initialization).

كما رأيت في عدة أمثلة سابقة، يتم تخزين قيمة في متغير باستخدام جملة الإسناد (assignment statement). على سبيل المثال، الجملة التالية تقوم بنسخ القيمة 12 إلى المتغير unitsSold:

unitsSold = 12;

يُسمى الرمز '=' عامل الإسناد (assignment operator). العوامل (operators) تقوم بتنفيذ عمليات على البيانات. البيانات التي تعمل عليها العوامل تُسمى المعاملات (operands). لعامل التخصيص معاملان. في الجملة السابقة، المعاملات هما 'unitsSold' و '12'.

جملة الإسناد تأخذ قيمة المعامل الأيمن (rvalue) وتضعها في موقع الذاكرة الخاص بالكائن الذي يُحدده المعامل الأيسر (lvalue). يمكنك أيضًا تعيين قيم للمتغيرات كجزء من تعريفها. تُسمى هذه العملية التهيئة (initialization). يوضح البرنامج 2-20 كيفية القيام بذلك.

Program 2-20

```
1 // This program shows variable initialization.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int month = 2, days = 28;
8
9     cout << "Month " << month << " has " << days << " days.\n";
10    return 0;
11 }
```

Program Output

Month 2 has 28 days.

Declaring Variables with the auto Key Word

أدخلت لغة C++11 طريقة بديلة لتعريف المتغيرات باستخدام الكلمة المفتاحية `auto` وقيمة التهيئة. إليك مثالاً:

```
auto amount = 100;
```

لاحظ أن هذه الجملة تستخدم الكلمة `auto` بدلاً من نوع بيانات محدد. الكلمة المفتاحية `auto` تخبر المترجم (compiler) بتحديد نوع بيانات المتغير من قيمة التهيئة. في هذا المثال، قيمة التهيئة 100 هي من نوع `int`، لذا سيكون المتغير `amount` من نوع `int`.
إليك أمثلة أخرى:

```
auto interestRate = 12.0;
auto stockCode = 'D';
auto customerNum = 459L;
```

في هذه الأمثلة:

- `interestRate` سيتم تحديده كـ `double` لأن القيمة 12.0 هي من نوع `double`.
- `stockCode` سيتم تحديده كـ `char` لأن القيمة 'D' هي من نوع `char`.
- `customerNum` سيتم تحديده كـ `long` لأن القيمة 459L هي من نوع `long`.

2.13 Scope

المفهوم: نطاق المتغير (scope) هو الجزء من البرنامج الذي يمكنه الوصول إلى المتغير.

القاعدة الأولى التي يجب أن تتعلمها حول النطاق هي أن المتغير لا يمكن استخدامه في أي جزء من البرنامج قبل تعريفه. يوضح البرنامج 2-21 هذه القاعدة.

Program 2-21

```
1 // This program can't find its variable.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     cout << value; // ERROR! value not defined yet!
8
9     int value = 100;
10    return 0;
11 }
```

البرنامج لن يعمل لأن السطر 7 يحاول إرسال محتويات المتغير 'value' إلى 'cout' قبل تعريف المتغير. يقوم المترجم (compiler) بقراءة برنامجك من الأعلى إلى الأسفل. إذا واجه عبارة تستخدم متغيراً قبل تعريفه، سيؤدي ذلك إلى حدوث خطأ. لتصحيح البرنامج، يجب وضع تعريف المتغير قبل أي عبارة تستخدمه.

2.14 Arithmetic Operators

المفهوم: هناك العديد من العوامل (operators) المستخدمة لمعالجة القيم العددية وتنفيذ العمليات الحسابية. تقدم لغة C++ مجموعة واسعة من العوامل لمعالجة البيانات. بشكل عام، هناك ثلاثة أنواع من العوامل:

1. العوامل الأحادية (Unary): تعمل على معامل واحد (operand).

2. العوامل الثنائية (Binary): تعمل على معاملين.

3. العوامل الثلاثية (Ternary): تعمل على ثلاث معاملات.

هذه المصطلحات تعكس عدد المعاملات التي يتطلبها العامل.

Table 2-9 Fundamental Arithmetic Operators

Operator	Meaning	Type	Example
+	Addition	Binary	total = cost + tax;
-	Subtraction	Binary	cost = total - tax;
*	Multiplication	Binary	tax = cost * rate;
/	Division	Binary	salePrice = original / 2;
%	Modulus	Binary	remainder = value % 3;

في لغة C++ ، تُستخدم العوامل (Operators) لإجراء عمليات مختلفة على البيانات. فيما يلي أمثلة على الأنواع الثلاثة من العوامل:

a. العوامل الأحادية: (Unary Operators)

تعمل هذه العوامل على معامل واحد فقط. من الأمثلة الشائعة:

- عامل الزيادة: (++) يزيد قيمة المتغير بواحد.
- عامل النقصان: (--) ينقص قيمة المتغير بواحد.

- عامل النفي: (-) يُغير إشارة القيمة.
- عامل العكس المنطقي: (!) يعكس القيمة المنطقية (من true إلى false والعكس)

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int a = 5;

    // عامل الزيادة
    ++a; // a أصبح 6
    cout << "After increment: " << a << endl;

    // عامل النقصان
    --a; // a أصبح 5
    cout << "After decrement: " << a << endl;

    // عامل النفي
    int b = -a; // b أصبح -5
    cout << "After negation: " << b << endl;

    // عامل العكس المنطقي
    bool flag = true;
    cout << "After logical NOT: " << !flag << endl; // الناتج: false

    return 0;
}
```

2. العوامل الثنائية: (Binary Operators)

تعمل هذه العوامل على معاملين. من الأمثلة الشائعة:

- عامل الجمع: (+) يجمع بين قيمتين.
- عامل الطرح: (-) يطرح قيمة من أخرى.
- عامل الضرب: (*) يضرب قيمتين.
- عامل القسمة: (/) يقسم قيمة على أخرى.
- عامل الباقي: (%) يعطي باقي القسمة.

```
#include <iostream>
using namespace std;
```

```
int main()
```



```

{
    int x = 10, y = 3;

    // عامل الجمع
    cout << "Sum: " << x + y << endl; // الناتج: 13

    // عامل الطرح
    cout << "Difference: " << x - y << endl; // الناتج: 7

    // عامل الضرب
    cout << "Product: " << x * y << endl; // الناتج: 30

    // عامل القسمة
    cout << "Quotient: " << x / y << endl; // الناتج: 3

    // عامل الباقي
    cout << "Remainder: " << x % y << endl; // الناتج: 1

    return 0;
}

```

3. العوامل الثلاثية (Ternary Operators):

تعمل هذه العوامل على ثلاث معاملات. العامل الثلاثي الوحيد في C++ هو العامل الشرطي. (?:)

- العامل الشرطي (?:) يعمل كاختصار لـ if-else إذا كان الشرط صحيحًا، يتم تنفيذ التعبير الأول، وإلا يتم تنفيذ التعبير الثاني.

```

#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 20;

    // العامل الشرطي
    int max = (a > b) ? a : b;
    cout << "Maximum value: " << max << endl; // الناتج: 20

    return 0;
}

```

القسمة الصحيحة (Integer Division):

عندما يكون كلا المعاملين في عبارة القسمة أعدادًا صحيحة (integers)، فإن الناتج سيكون قسمة صحيحة. هذا يعني أن نتيجة القسمة ستكون عددًا صحيحًا أيضًا، وإذا كان هناك باقي، فسيتم تجاهله. على سبيل المثال، انظر إلى الكود التالي:

```
double number;  
number = 5 / 2;
```

Program 2-22

```
1 // This program calculates hourly wages, including overtime.  
2 #include <iostream>  
3 using namespace std;  
4  
5 int main()  
6 {  
7     double regularWages,           // To hold regular wages  
8         basePayRate = 18.25,       // Base pay rate  
9         regularHours = 40.0,       // Hours worked less overtime  
10    overtimeWages,                 // To hold overtime wages  
11    overtimePayRate = 27.78,        // Overtime pay rate  
12    overtimeHours = 10,             // Overtime hours worked  
13    totalWages;                     // To hold total wages  
14  
15    // Calculate the regular wages.  
16    regularWages = basePayRate * regularHours;  
17  
18    // Calculate the overtime wages.  
19    overtimeWages = overtimePayRate * overtimeHours;  
20  
21    // Calculate the total wages.  
22    totalWages = regularWages + overtimeWages;  
23  
24    // Display the total wages.  
25    cout << "Wages for this week are $" << totalWages << endl;  
26    return 0;  
27 }
```

Program Output

Wages for this week are \$1007.8

يقوم هذا الكود بقسمة 5 على 2 وتعيين الناتج إلى المتغير number. ما الذي سيتم تخزينه في المتغير number؟ قد تفترض أن القيمة 2.5 سيتم تخزينها في number لأن هذه هي النتيجة التي تظهرها الآلة الحاسبة عند قسمة 5 على 2. ومع ذلك، هذا ليس ما يحدث عند تنفيذ الكود السابق في لغة C++ لأن العددين 5 و2 هما عددين صحيحين، سيتم تجاهل الجزء الكسري من الناتج (أو اقتطاعه). نتيجة لذلك، سيتم تعيين القيمة 2 إلى المتغير number.

في الكود السابق، لا يهم أن المتغير number تم تعريفه كـ double لأن الجزء الكسري من الناتج يتم تجاهله قبل حدوث عملية التعيين. حتى تُرجع عملية القسمة قيمة عشرية (floating-point)، يجب أن يكون أحد المعاملين من نوع بيانات عشري. على سبيل المثال، يمكن كتابة الكود السابق كما يلي:

```
double number;  
number = 5.0 / 2;
```

في هذا الكود، يتم التعامل مع العدد 5.0 كعدد عشري، لذا فإن عملية القسمة سترجع قيمة عشرية. نتيجة القسمة في هذه الحالة ستكون 2.5.

انظر البرنامجين التاليين ولاحظ أهمية معامل باقي القسمة:

Program 2-25

```
1 // This program extracts the rightmost digit of a number.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int number = 12345;
8     int rightMost = number % 10;
9
10    cout << "The rightmost digit in "
11         << number << " is "
12         << rightMost << endl;
13
14    return 0;
15 }
```

Program Output

The rightmost digit in 12345 is 5

Program 2-26

```
1 // This program converts seconds to minutes and seconds.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // The total seconds is 125.
8     int totalSeconds = 125;
9
10    // Variables for the minutes and seconds
11    int minutes, seconds;
12
13    // Get the number of minutes.
14    minutes = totalSeconds / 60;
15
16    // Get the remaining seconds.
17    seconds = totalSeconds % 60;
18
19    // Display the results.
20    cout << totalSeconds << " seconds is equivalent to:\n";
21    cout << "Minutes: " << minutes << endl;
22    cout << "Seconds: " << seconds << endl;
23    return 0;
24 }
```

Program Output

125 seconds is equivalent to:
Minutes: 2
Seconds: 5

2.15 Comments

المفهوم: التعليقات هي ملاحظات توضيحية تُستخدم لتوثيق الأسطر أو الأقسام في البرنامج. التعليقات هي جزء من البرنامج، لكن المترجم (compiler) يتجاهلها. الغرض منها هو مساعدة الأشخاص الذين قد يقرأون الكود المصدري (Source Code).

- **Single-Line Comments**

Program 2-27 shows that comments may be placed liberally throughout a program.

Program 2-27

```
1 // PROGRAM: PAYROLL.CPP
2 // Written by Herbert Dorfmann
3 // This program calculates company payroll
4 // Last modification: 8/20/2020
5 #include <iostream>
6 using namespace std;
7
8 int main()
9 {
10     double payRate;    // Holds the hourly pay rate
11     double hours;      // Holds the hours worked
12     int employNumber;  // Holds the employee number
    (The remainder of this program is left out.)
```

- **Multi-Line Comments**

Program 2-28

```
1 /*
2     PROGRAM: PAYROLL.CPP
3     Written by Herbert Dorfmann
4     This program calculates company payroll
5     Last modification: 8/20/2020
6 */
7
8 #include <iostream>
9 using namespace std;
10
11 int main()
12 {
13     double payRate;    // Holds the hourly pay rate
14     double hours;      // Holds the hours worked
15     int employNumber;  // Holds the employee number
    (The remainder of this program is left out.)
```

تذكر النصائح التالية عند استخدام التعليقات متعددة الأسطر:

- كن حذرًا من عدم عكس رمز البداية مع رمز النهاية.
- تأكد من عدم نسيان رمز النهاية

كلا الخطأين، قد يكون من الصعب اكتشافهما وستمنع البرنامج من الترجمة (compilation) بشكل صحيح.

2.16 Named Constants

المفهوم: يمكن إعطاء القيم الثابتة (Literals) أسماء تمثلها بشكل رمزي في البرنامج.

Program 2-29

```
1 // This program calculates the circumference of a circle.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     // Constants
8     const double PI = 3.14159;
9     const double DIAMETER = 10.0;
10
11     // Variable to hold the circumference
12     double circumference;
13
14     // Calculate the circumference.
15     circumference = PI * DIAMETER;
16
```

Program 2-29 (continued)

```
17 // Display the circumference.
18 cout << "The circumference is: " << circumference << endl;
19 return 0;
20 }
```

Program Output

The circumference is: 31.4159

لنلقي نظرة أقرب على البرنامج. في السطر 8، يتم تعريف ثابت من النوع double باسم PI، ويتم تهيئته بالقيمة 3.14159 سيتم استخدام هذا الثابت لقيمة باي (π) في حسابات البرنامج. في السطر 9، يتم تعريف ثابت آخر من النوع double باسم DIAMETER، ويتم تهيئته بالقيمة 10 سيتم استخدام هذا الثابت لتمثيل قطر الدائرة. في السطر 12، يتم تعريف متغير من النوع double باسم circumference، والذي سيستخدم لتخزين محيط الدائرة. في السطر 15، يتم حساب محيط الدائرة عن طريق ضرب PI في DIAMETER يتم إسناد نتيجة الحساب إلى المتغير circumference في السطر 18، يتم عرض محيط الدائرة.

2.17 Programming Style

المفهوم: يشير أسلوب البرمجة (Programming Style) إلى الطريقة التي يستخدم بها المبرمج المعارف (identifiers)، المسافات، علامات التبويب (tabs)، الأسطر الفارغة، ورموز الترقيم لترتيب الكود المصدري للبرنامج بشكل مرئي. هذه بعض عناصر أسلوب البرمجة، ولكنها ليست كلها.

Program 2-30

```
1 #include <iostream>
2 using namespace std;int main(){double shares=220.0;
3 double avgPrice=14.67;cout<<"There were "<<shares
4 <<" shares sold at $"<<avgPrice<<" per share.\n";
5 return 0;}
```

Program Output

There were 220 shares sold at \$14.67 per share.

على الرغم من أن البرنامج صحيح من الناحية التركيبية (أي لا يخالف أي قواعد في لغة C++)، إلا أنه من الصعب جدًا قراءته. يُظهر البرنامج 2-31 نفس البرنامج مكتوبًا بأسلوب أكثر منطقية وسهولة في الفهم.

Program 2-31

```
1 // This example is much more readable than Program 2-30.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     double shares = 220.0;
8     double avgPrice = 14.67;
9
10    cout << "There were " << shares << " shares sold at $";
11    cout << avgPrice << " per share.\n";
12    return 0;
13 }
```

Program Output

There were 220 shares sold at \$14.67 per share.



NOTE: Although you are free to develop your own style, you should adhere to common programming practices. By doing so, you will write programs that visually make sense to other programmers.

انتهت المحاضرة الثانية